

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВА-
ТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

Практическая работа 5

По дисциплине Проектирование и реализация баз данных

Тема работы Исследование транзакций, блокировок и уровней изоляции в PostgreSQL

Обучающийся Рекун Ирина Данииловна

Факультет прикладной информатики

Группа K3220

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся

(дата)

(подпись)

Рекун И.Д

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Войтюк Т.Е

(Ф.И.О.)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
Задание 1. Изучение транзакций COMMIT и ROLLBACK в PostgreSQL	4
Задание 2. Поиск и обнаружение блокировок	9
Задание 3. Уровни изоляции READ UNCOMMITTED и READ COMMITTED	14
Задание 5. Уровень изоляции SERIALIZABLE	19
Выводы и анализ результатов работы	21

ВВЕДЕНИЕ

Цель работы: Изучить механизм транзакций в СУБД PostgreSQL, принципы фиксации и отката изменений, работу точек сохранения, систему блокировок, а также влияние различных уровней изоляции транзакций на поведение параллельных операций с данными.

Задачи, решаемые при выполнении работы:

1. Освоить создание обычных представлений (VIEW) с использованием графического интерфейса pgadmin 4.
2. Изучить создание представлений с помощью DDL-оператора CREATE VIEW.
3. Исследовать влияние представлений на возможность изменения базовых таблиц.
4. Создать и протестировать материализованное представление (MATERIALIZED VIEW).
5. Изучить различия между обычными и материализованными представлениями.
6. Создать и протестировать скалярную пользовательскую функцию.
7. Разработать собственную скалярную функцию с логикой ветвления.
8. Создать и протестировать хранимую процедуру.
9. Разработать собственную хранимую процедуру с несколькими режимами работы.

Задание 1. Изучение транзакций COMMIT и ROLLBACK в PostgreSQL

Необходимо запустить Query Tool (Редактор запросов) в PgAdmin и в окне запроса ввести код, представленный на рисунке 1.1. Он содержит блок транзакции, который начинается с BEGIN, и оператор COMMIT, который завершает транзакцию и фиксирует в БД изменения, произведенные во время её выполнения. Выполняется запрос для просмотра значения заработной платы сотрудника до изменений, затем запускается транзакция, выполняется обновление и производится фиксация изменений.

```
1 SELECT 'Before' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"
2 FROM "EmployeesDepartments"."EMPLOYEES"
3 WHERE "EMPLOYEE_ID" = 101;
4
5 BEGIN;
6
7 UPDATE "EmployeesDepartments"."EMPLOYEES"
8 SET "SALARY" = "SALARY" * 1.1
9 WHERE "EMPLOYEE_ID" = 101;
10
11 SELECT 'During' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"
12 FROM "EmployeesDepartments"."EMPLOYEES"
13 WHERE "EMPLOYEE_ID" = 101;
14
15 COMMIT;
16
17 SELECT 'After' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"
18 FROM "EmployeesDepartments"."EMPLOYEES"
19 WHERE "EMPLOYEE_ID" = 101;
```

Рисунок 1.1 – Объявление блока транзакции и её фиксация.

После необходимо выполнить данный код последовательно, зафиксировать результаты значения до (Рисунок 1.2), во время (Рисунок 1.3) и после транзакции (Рисунок 1.4)

	stage text	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	SALARY numeric (8,2)
1	Before	101	Neena	Kochhar	18700.00

Рисунок 1.2 – Результат до начала транзакции (Before).

	stage text	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	SALARY numeric (8,2)
1	During	101	Neena	Kochhar	22627.00

Рисунок 1.3 – Результат внутри транзакции до COMMIT

	stage	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
	text	[PK] integer	character varying (20)	character varying (25)	numeric (8,2)
1	After	101	Neena	Kochhar	22627.00

Рисунок 1.4 – Результат после фиксации транзакции

Таким образом, до транзакции отображается исходное значение зарплаты, внутри транзакции видно увеличенное значение, после выполнения COMMIT изменения окончательно сохраняются в базе данных.

Теперь исследуем поведение отката транзакции ROLLBACK для этого был введен код, представленный на рисунке 1.5.

```
SELECT 'Before' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"
FROM "EmployeesDepartments"."EMPLOYEES"
WHERE "EMPLOYEE_ID" <= 103
ORDER BY 2;

BEGIN;

UPDATE "EmployeesDepartments"."EMPLOYEES"
SET "SALARY" = "SALARY" * 1.1
WHERE "EMPLOYEE_ID" <= 103;

SELECT 'Within transaction' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"
FROM "EmployeesDepartments"."EMPLOYEES"
WHERE "EMPLOYEE_ID" <= 103
ORDER BY 2;

ROLLBACK;

SELECT 'After' AS stage, "EMPLOYEE_ID", "FIRST_NAME", "LAST_NAME", "SALARY"
FROM "EmployeesDepartments"."EMPLOYEES"
WHERE "EMPLOYEE_ID" <= 103
ORDER BY 2;
```

Рисунок 1.5 – Откат транзакции

Далее данный код выполняется последовательно, фиксируются результаты значения до (Рисунок 1.6), во время (Рисунок 1.7) и после транзакции (Рисунок 1.8)

	stage	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
	text	[PK] integer	character varying (20)	character varying (25)	numeric (8,2)
1	Before	100	Steven	King	24000.00
2	Before	101	Neena	Kochhar	22627.00
3	Before	102	Lex	De Haan	17000.00
4	Before	103	Alexander	Hunold	9000.00

Рисунок 1.6 – Значения заработной платы сотрудников до начала транзакции.

Было выполнено обновление заработной платы путём увеличения значения поля SALARY на 10%.

	stage text	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	SALARY numeric (8,2)
1	During transacti...	100	Steven	King	26400.00
2	During transacti...	101	Neena	Kochhar	24889.70
3	During transacti...	102	Lex	De Haan	18700.00
4	During transacti...	103	Alexander	Hunold	9900.00

Рисунок 1.7 – Значения заработной платы сотрудников внутри транзакции до выполнения ROLLBACK

После этого была выполнена команда ROLLBACK, в результате которой все изменения, произведённые внутри транзакции, были отменены.

	stage text	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	SALARY numeric (8,2)
1	After ROLLBA...	100	Steven	King	24000.00
2	After ROLLBA...	101	Neena	Kochhar	22627.00
3	After ROLLBA...	102	Lex	De Haan	17000.00
4	After ROLLBA...	103	Alexander	Hunold	9000.00

Рисунок 1.8 – Значения заработной платы сотрудников после выполнения ROLLBACK

Таким образом, было продемонстрировано, что оператор ROLLBACK отменяет все изменения, выполненные в рамках текущей транзакции, если они не были зафиксированы с помощью COMMIT. Это обеспечивает целостность данных при возникновении ошибок или необходимости отмены изменений.

Использование точек сохранения SAVEPOINT

Вначале необходимо было зафиксировать исходное состояние данных до выполнения транзакции (Рисунок 1.9).

1

2

3

4

5

6

7

8

```
SELECT 'Before transaction' AS stage,  
      "EMPLOYEE_ID",  
      "FIRST_NAME",  
      "LAST_NAME",  
      "SALARY"  
FROM "EmployeesDepartments"."EMPLOYEES"  
WHERE "EMPLOYEE_ID" = 104;
```

Data OutputСообщенияNotifications

Showing rows: 1 to 1

	stage text	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	SALARY numeric (8,2)
1	Before transacti...	104	Bruce	Ernst	6000.00

Рисунок 1.9 – Исходные данные сотрудника до начала транзакции

Запускается транзакция и создаётся первая точка сохранения (Рисунок 1.10). Точка сохранения sp1 фиксирует состояние транзакции до выполнения первого изменения.

```
8 BEGIN;  
9  
10 SAVEPOINT sp1;  
11
```

Data Output Сообщения Notifications

SAVEPOINT

Запрос завершён успешно, время выполнения: 169 msec.

Рисунок 1.10 – Точка сохранения sp1.

После первой точки сохранения изменяется значение заработной платы (Рисунок 1.11).

```
1 UPDATE "EmployeesDepartments"."EMPLOYEES"  
2 SET "SALARY" = 3000  
3 WHERE "EMPLOYEE_ID" = 104;  
4 SELECT 'After SAVEPOINT sp1' AS stage,  
5        "EMPLOYEE_ID",  
6        "FIRST_NAME",  
7        "LAST_NAME",  
8        "SALARY"  
9 FROM "EmployeesDepartments"."EMPLOYEES"  
10 WHERE "EMPLOYEE_ID" = 104;
```

Data Output Сообщения Notifications

+

SQL

Showing rows: 1 to 1

	stage text	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	SALARY numeric (8,2)
1	After SAVEPOINT s...	104	Bruce	Ernst	3000.00

Рисунок 1.11 — Изменение данных после точки сохранения sp1 (внутри транзакции)

Далее создаётся вторая точка сохранения sp2, которая фиксирует состояние данных после первого изменения. После второй точки сохранения выполняется повторное обновление данных. Внутри транзакции заработная плата изменена на 10000 (Рисунок 1.12).

1

2

3

4

5

6

7

8

9

10

11

```
UPDATE "EmployeesDepartments"."EMPLOYEES"
SET "SALARY" = 10000
WHERE "EMPLOYEE_ID" = 104;
SELECT 'After SAVEPOINT sp2' AS stage,
       "EMPLOYEE_ID",
       "FIRST_NAME",
       "LAST_NAME",
       "SALARY"
FROM "EmployeesDepartments"."EMPLOYEES"
WHERE "EMPLOYEE_ID" = 104;
```

Data Output

Сообщения

Notifications

≡

+

📄

▼

📋

▼

🗑️

🔄

⬇️

📈

SQL

Showing rows: 1 to 1

✎

Page N

	stage text	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	SALARY numeric (8,2)
1	After SAVEPOINT s...	104	Bruce	Ernst	10000.00

Рисунок 1.12 – Повторное изменение данных после SAVEPOINT sp2

Выполняется частичный откат транзакции и проверка данных после отката (Рисунок 1.13). Таким образом последнее изменение отменено, значение заработной платы вернулось к состоянию на момент создания sp2.

1

SELECT 'After ROLLBACK TO sp2' AS stage,

2

"EMPLOYEE_ID",

3

"FIRST_NAME",

4

"LAST_NAME",

5

"SALARY"

6

FROM "EmployeesDepartments"."EMPLOYEES"

7

WHERE "EMPLOYEE_ID" = 104;

8

Data Output

Сообщения

Notifications

≡

📄

▼

📋

▼

🗑️

🔄

⬇️

📈

SQL

Showing rows: 1 to 1

✎

 Page 1

	stage text	EMPLOYEE_ID [PK] integer	FIRST_NAME character varying (20)	LAST_NAME character varying (25)	SALARY numeric (8,2)
1	After ROLLBACK TO s...	104	Bruce	Ernst	3000.00

Рисунок 1.13 – Состояние данных после ROLLBACK TO SAVEPOINT sp2

Выполняется дополнительное обновление и фиксация транзакции (Рисунок 1.14). Итоговое значение заработной платы сохранено в базе данных.

1

UPDATE "EmployeesDepartments"."EMPLOYEES"

2

SET "SALARY" = "SALARY" + 100

3

WHERE "EMPLOYEE_ID" = 104;

4

5

COMMIT;

6

SELECT 'After COMMIT' AS stage,

7

"EMPLOYEE_ID",

8

"FIRST_NAME",

9

"LAST_NAME",

10

"SALARY"

11

FROM "EmployeesDepartments"."EMPLOYEES"

12

WHERE "EMPLOYEE_ID" = 104;

13

Data Output

Сообщения

Notifications

≡

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 1

✎

	stage	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
	text	[PK] integer	character varying (20)	character varying (25)	numeric (8,2)
1	After COMM...	104	Bruce	Ernst	3200.00

Рисунок 1.14 – Итоговое состояние данных после выполнения COMMIT

В ходе выполнения задания было установлено, что точки сохранения SAVEPOINT позволяют выполнять частичный откат транзакции без отмены всех изменений. Использование команд SAVEPOINT и ROLLBACK TO SAVEPOINT обеспечивает гибкое управление изменениями данных внутри одной транзакции.

Задание 2. Поиск и обнаружение блокировок

Создаём тестовую таблицу public.t1 и заполняем её начальными данными.

```

1
2 CREATE TABLE public.t1 (
3     id int PRIMARY KEY,
4     price numeric(10,2)
5 );
6
7 INSERT INTO public.t1 VALUES
8 (1, 10.00),
9 (2, 20.00),
10 (3, 30.00);
11

```

Рисунок 2.1 – Создание тестовой таблицы и наполнение её данными

Далее необходимо создать три активные сессии к одной и той же БД. Для этого открываем три независимые вкладки Query Tool в pgAdmin и

переименовываем их на: Connection1, Connection2, Connection3 и фиксируем, что все три сессии подключены к одной и той же базе данных, что позволяет моделировать параллельную работу пользователей (Рисунок 2.2)

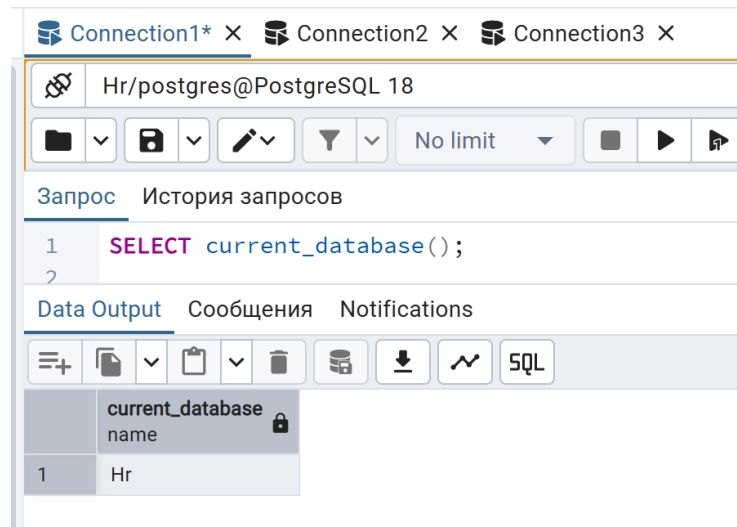


Рисунок 2.2 – три независимые вкладки Query Tool

В первой сессии начинаем транзакцию и обновляем строку с `id = 2`, не выполняя COMMIT (Рисунок 2.3).

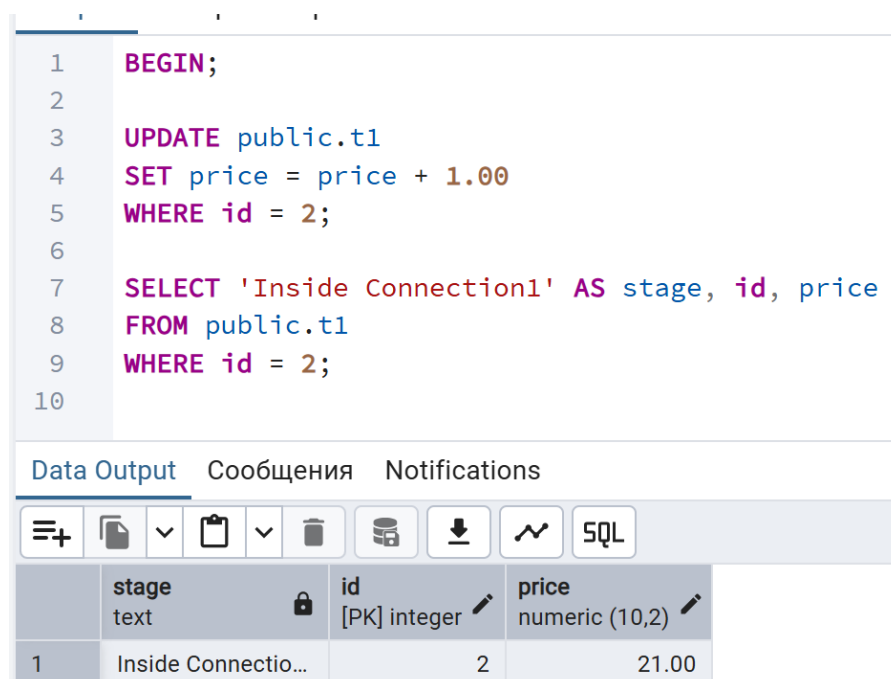


Рисунок 2.3 — Транзакция в первой сессии (установка блокировки строки)

Таким образом, строка `id = 2` изменена внутри транзакции, устанавливается эксклюзивная блокировка строки и пока транзакция не зафиксирована, блокировка удерживается.

Далее во второй сессии пытаемся изменить ту же строку. Но `UPDATE` зависает, так как строка заблокирована транзакцией из `Connection1`. Вторая сессия ожидает освобождения блокировки, обновление не выполняется (Рисунок 2.4).

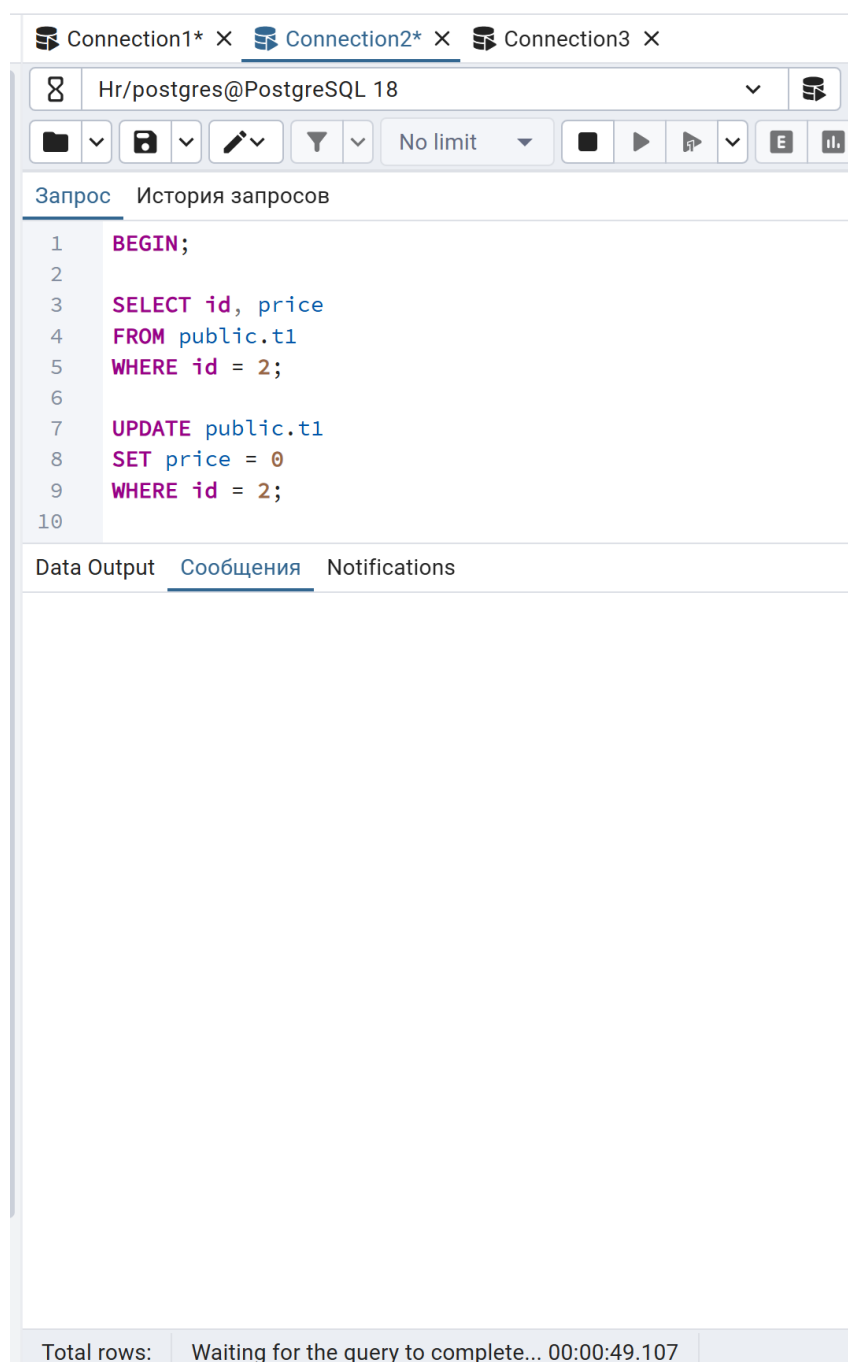
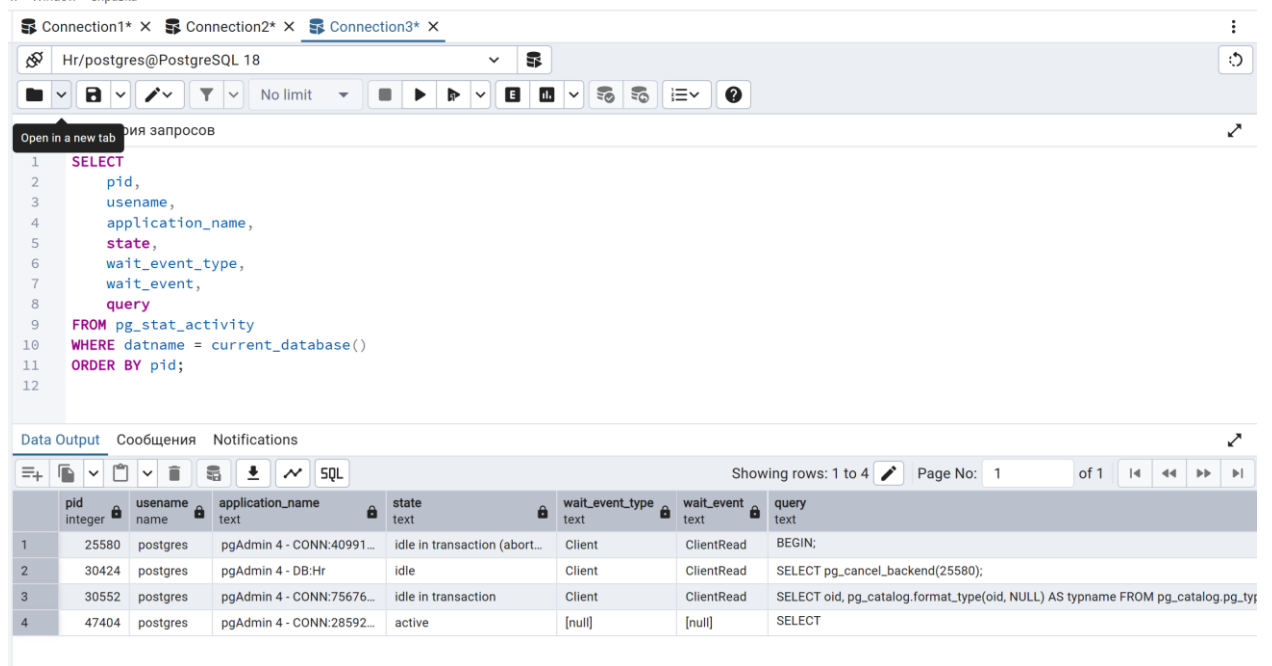


Рисунок 2.4 — Попытка обновления заблокированной строки во второй сессии

Выполняем анализ блокировок через `pg_stat_activity` (Рисунок 2.5).



The screenshot shows a PostgreSQL client window with three tabs: Connection1*, Connection2*, and Connection3*. The active tab is Connection3*. The query editor shows a SQL query to select from `pg_stat_activity`. The results pane shows a table with 4 rows and 7 columns: `pid`, `username`, `application_name`, `state`, `wait_event_type`, `wait_event`, and `query`.

	pid	username	application_name	state	wait_event_type	wait_event	query
1	25580	postgres	pgAdmin 4 - CONN:40991...	idle in transaction (abort...	Client	ClientRead	BEGIN;
2	30424	postgres	pgAdmin 4 - DB:Hr	idle	Client	ClientRead	SELECT pg_cancel_backend(25580);
3	30552	postgres	pgAdmin 4 - CONN:75676...	idle in transaction	Client	ClientRead	SELECT oid, pg_catalog.format_type(oid, NULL) AS typename FROM pg_catalog.pg_ty...
4	47404	postgres	pgAdmin 4 - CONN:28592...	active	[null]	[null]	SELECT

Рисунок 2.5 — Анализ активных сессий с помощью `pg_stat_activity`

Замечаем, что Connection1 — `idle in transaction` (удерживает блокировку), а Connection2 — ожидает блокировку (Lock).

Далее необходимо просмотреть блокировки через `pg_locks` (Рисунок 2.6). Этот запрос выводит сводную информацию сразу из двух системных представлений: `pg_locks`, где фиксируются сведения о блокировках, и `pg_stat_activity`, отображающего состояние серверных процессов. Таким образом можно увидеть, какой процесс удерживает или ожидает конкретную блокировку, какое именно действие он выполняет и в каком состоянии сейчас находится.

Запрос История запросов

```
1  SELECT
2    l.pid,
3    a.application_name,
4    a.state,
5    l.locktype,
6    l.relation::regclass AS locked_relation,
7    l.mode,
8    l.granted
9  FROM pg_locks l
10 LEFT JOIN pg_stat_activity a ON a.pid = l.pid
11 WHERE a.datname = current_database()
12 ORDER BY l.pid;
```

Data Output Сообщения Notifications

Showing rows: 1 to 47

Page No: 1

	pid integer	application_name text	state text	locktype text	locked_relation regclass	mode text	granted boolean
1	30552	pgAdmin 4 - CONN:75676...	idle in transacti...	relation	t1	RowExclusiveLo...	true
2	30552	pgAdmin 4 - CONN:75676...	idle in transacti...	relation	pg_sequence	AccessShareLock	true
3	30552	pgAdmin 4 - CONN:75676...	idle in transacti...	relation	pg_depend	AccessShareLock	true
4	30552	pgAdmin 4 - CONN:75676...	idle in transacti...	relation	pg_attrdef	AccessShareLock	true
5	30552	pgAdmin 4 - CONN:75676...	idle in transacti...	relation	pg_constraint_conname_nsp_index	AccessShareLock	true
6	30552	pgAdmin 4 - CONN:75676...	idle in transacti...	relation	pg_namespace_nspname_index	AccessShareLock	true
7	30552	pgAdmin 4 - CONN:75676...	idle in transacti...	relation	pg_type_typname_nsp_index	AccessShareLock	true
8	30552	pgAdmin 4 - CONN:75676...	idle in transacti...	relation	t1_pkey	AccessShareLock	true
9	30552	pgAdmin 4 - CONN:75676...	idle in transacti...	relation	t1_pkey	RowExclusiveLo...	true
10	30552	pgAdmin 4 - CONN:75676...	idle in transacti...	relation	t1	AccessShareLock	true
11	30552	pgAdmin 4 - CONN:75676...	idle in transacti...	relation	pg_attrdef_adrelid_adnum_index	AccessShareLock	true
12	30552	pgAdmin 4 - CONN:75676...	idle in transacti...	relation	pg_depend_depender_index	AccessShareLock	true
13	30552	pgAdmin 4 - CONN:75676...	idle in transacti...	relation	pg_attrdef_oid_index	AccessShareLock	true
14	30552	pgAdmin 4 - CONN:75676...	idle in transacti...	relation	pg_description	AccessShareLock	true

Рисунок 2.6 — Анализ блокировок с помощью pg_locks

После возвращаемся в Connection1 и выполняем COMMIT; для завершения транзакций. После коммита блокировка снимается — и Connection2 сможет продолжить выполнение (его UPDATE завершится).

Снова выполняем запросы к pg_locks и pg_stat_activity, чтобы убедиться, что блокировки исчезли (Рисунки 2.7 – 2.8).

Запрос

История запросов

```
1 SELECT
2     pid,
3     username,
4     application_name,
5     state,
6     wait_event_type,
7     wait_event,
8     query
9 FROM pg_stat_activity
10 WHERE datname = current_database()
11 ORDER BY pid;
```

12

Data Output

Сообщения

Notifications

</

Рисунок 2.7 — Анализ активных сессий с помощью pg_stat_activity после COMMIT

1	SELECT
2	l.pid,
3	a.application_name,
4	a.state,
5	l.locktype,
6	l.relation::regclass AS locked_relation,
7	l.mode,
8	l.granted
9	FROM pg_locks l
10	LEFT JOIN pg_stat_activity a ON a.pid = l.pid
11	WHERE a.datname = current_database()
12	ORDER BY l.pid;
13	
14	

Data Output	Сообщения	Notifications
-------------	-----------	---------------

SQL	Showing rows: 1
-----	-----------------

	pid integer	application_name text	state text	locktype text	locked_relation regclass	mode text	granted boolean
1	47404	pgAdmin 4 - CONN:28592...	active	relation	pg_locks	AccessShareLo...	true
2	47404	pgAdmin 4 - CONN:28592...	active	relation	pg_stat_activity	AccessShareLo...	true
3	47404	pgAdmin 4 - CONN:28592...	active	virtualxid	[null]	ExclusiveLock	true
4	47404	pgAdmin 4 - CONN:28592...	active	relation	pg_database_datname_ind...	AccessShareLo...	true
5	47404	pgAdmin 4 - CONN:28592...	active	relation	pg_authid	AccessShareLo...	true
6	47404	pgAdmin 4 - CONN:28592...	active	relation	pg_authid_oid_index	AccessShareLo...	true
7	47404	pgAdmin 4 - CONN:28592...	active	relation	pg_database_oid_index	AccessShareLo...	true
8	47404	pgAdmin 4 - CONN:28592...	active	relation	pg_database	AccessShareLo...	true
9	47404	pgAdmin 4 - CONN:28592...	active	relation	pg_authid_rolname_index	AccessShareLo...	true

Рисунок 2.8 — Анализ блокировок с помощью pg_locks после COMMIT

Так, в ходе выполнения задания была создана ситуация блокировки строки при параллельной работе нескольких транзакций. Было показано, что незавершённая транзакция удерживает эксклюзивную блокировку, препятствуя изменениям со стороны других сессий. С помощью системных представлений pg_stat_activity и pg_locks были успешно выявлены блокирующие и ожидающие процессы. Полученные результаты подтверждают корректную работу механизма блокировок PostgreSQL и его средств диагностики.

Задание 3. Уровни изоляции READ UNCOMMITTED и READ COMMITTED

В первой сессии была запущена транзакция, в рамках которой выполнено чтение и изменение значения поля price для товара с идентификатором 3 (Рисунок 3.1). После выполнения оператора UPDATE новое значение становится доступным внутри текущей транзакции, однако изменения ещё не зафиксированы и недоступны другим сессиям.

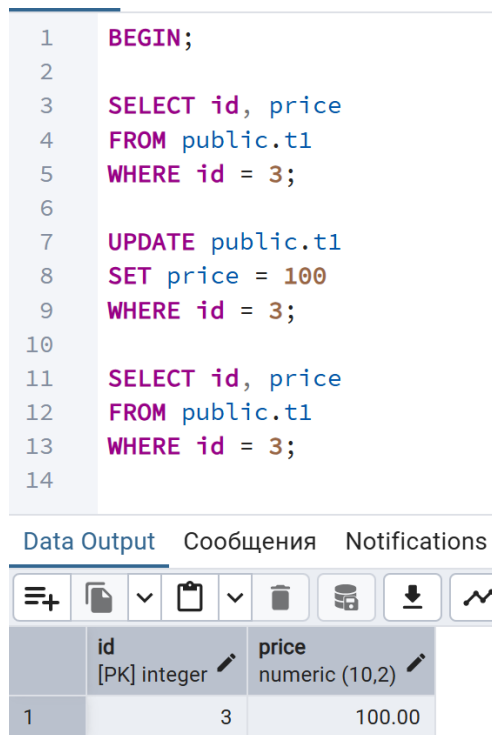


Рисунок 3.1 — Запуск транзакции и изменение данных в Session1

Во второй сессии была начата транзакция с уровнем изоляции READ COMMITTED (Рисунок 3.2). При выполнении операции чтения были получены старые данные, так как изменения, выполненные в первой сессии, на данный момент не были зафиксированы. Это подтверждает отсутствие грязного чтения в режиме READ COMMITTED.

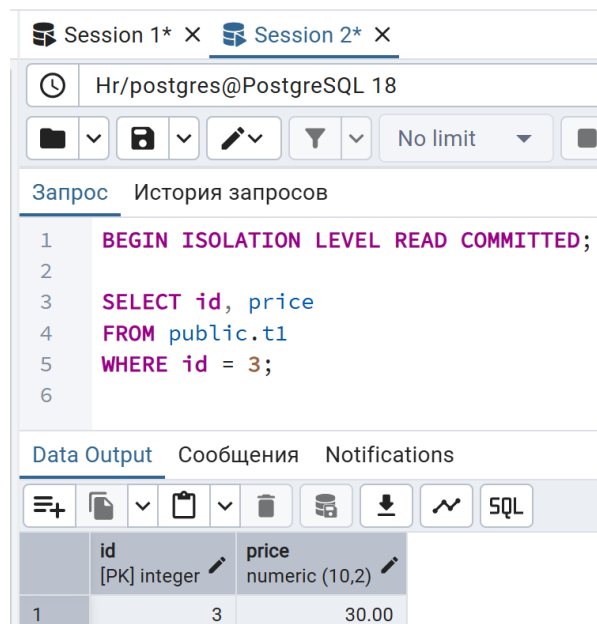


Рисунок 3.2 — Чтение данных в Session2 до фиксации транзакции

После выполнения команды COMMIT в первой сессии изменения были зафиксированы в базе данных, а установленные блокировки сняты.

После фиксации транзакции в первой сессии повторный запрос во второй сессии вернул обновлённое значение (Рисунок 3.3). Это демонстрирует особенность уровня READ COMMITTED: каждый оператор SELECT видит данные, зафиксированные на момент его выполнения.

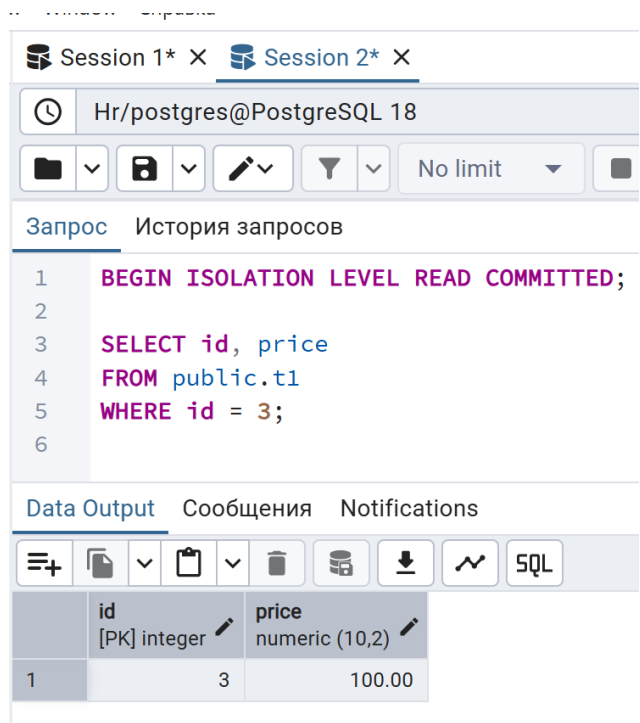


Рисунок 3.3 — Чтение данных в Session2 после COMMIT

Задание 4. Уровень изоляции REPEATABLE READ

В первой сессии была запущена транзакция с уровнем изоляции REPEATABLE READ. После начала транзакции выполнен запрос на чтение данных о товаре с идентификатором 3. При выполнении запроса чтения были получены данные, зафиксированные на момент начала транзакции. Данный снимок данных сохраняется неизменным до завершения транзакции. Данная транзакция представлена на рисунке 4.1.

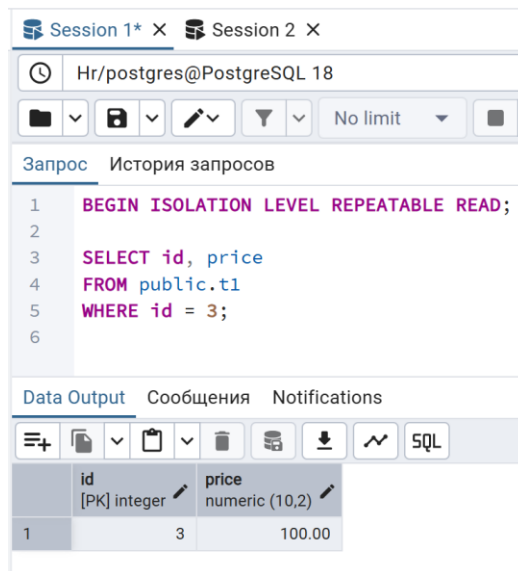


Рисунок 4.1 — Запуск транзакции с уровнем изоляции REPEATABLE READ

В окне Session2 был выполнен код, представленный на рисунке 4.2, чтобы попытаться изменить цену товара 3.

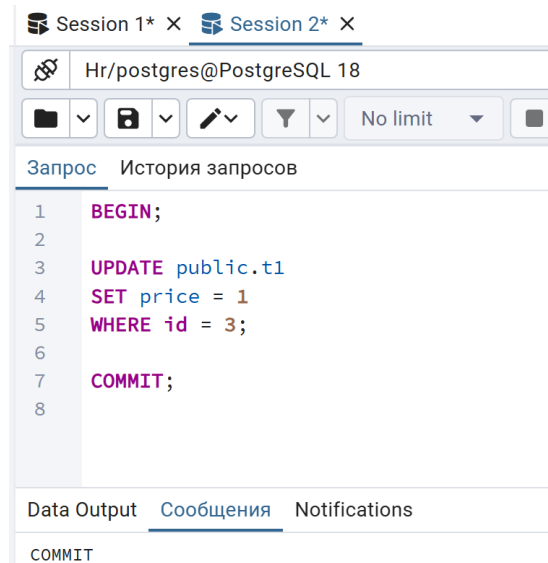


Рисунок 4.2 — Изменение цены товара в Session2

Изменения были успешно зафиксированы, однако они не повлияли на данные, видимые в транзакции Session1, так как она использует ранее зафиксированный снимок данных. Поэтому повторное выполнение запроса чтения в первой сессии вернуло те же данные, что и при первом запросе. Это подтверждает, что при уровне изоляции REPEATABLE READ все операции чтения в рамках одной транзакции работают с неизменным снимком данных. После завершения транзакции изменения, выполненные в других сессиях, становятся видимыми.

Далее была запущена транзакция с уровнем изоляции REPEATABLE READ и выполнен запрос чтения строк по условию price = 1 (Рисунок 4.3).

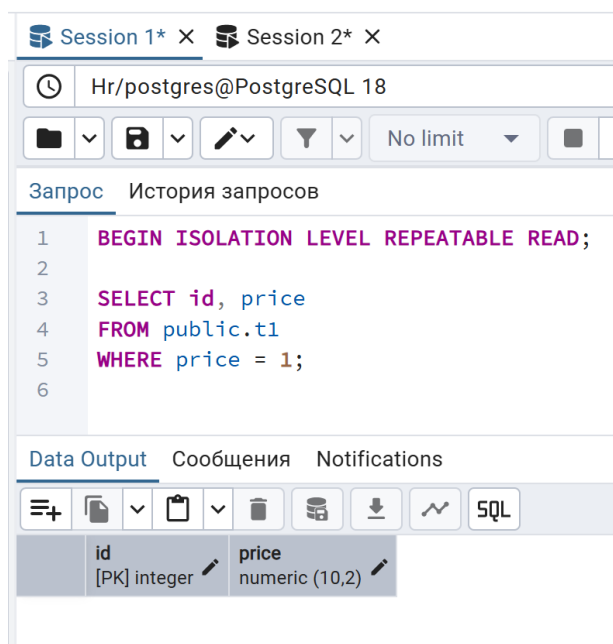


Рисунок 4.3 — Запуск транзакции с уровнем изоляции REPEATABLE READ и первичное чтение данных

В окне Session2 был написан код, чтобы добавить ещё один товар с идентификатором 4 и ценой 1 (Рисунок 4.4).

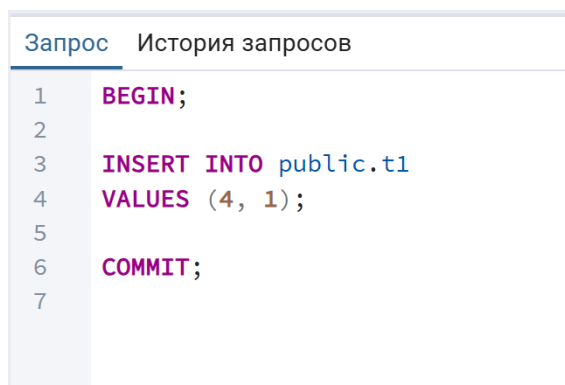


Рисунок 4.4 – Добавление новой строки в Session2

Повторное выполнение запроса в первой сессии вернуло тот же набор строк, что и при первом чтении (Рисунок 4.5). Это подтверждает отсутствие фантомного чтения при уровне изоляции REPEATABLE READ.

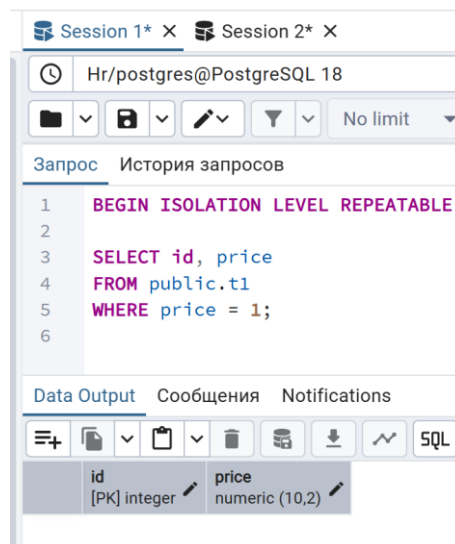


Рисунок 4.5 — Повторное чтение данных и завершение транзакции в Session1

Задание 5. Уровень изоляции SERIALIZABLE

Уровень изоляции `SERIALIZABLE` в PostgreSQL обеспечивает максимальную изоляцию транзакций, гарантируя, что результаты выполнения параллельных транзакций эквивалентны некоторому последовательному (последовательно выполняемому) порядку их выполнения.

Для демонстрации поведения транзакций в режиме `SERIALIZABLE` используются две активные сессии: Session1 и Session2. В первой сессии запускается транзакция с уровнем изоляции `SERIALIZABLE` и выполняется обновление строки с `id = 1` (Рисунок 5.1).

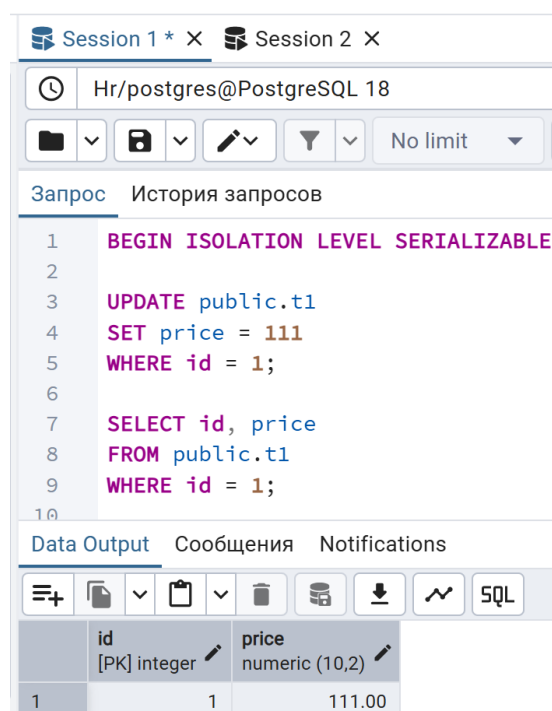


Рисунок 5.1 — Запуск транзакции в режиме SERIALIZABLE в Session 1

Транзакция успешно обновила строку и была получена обновленная цена\а для товара 1.

В окне Session2 необходимо попытаться также изменить цену товара 1, для этого был выполнен код, представленный на рисунке 5.2.

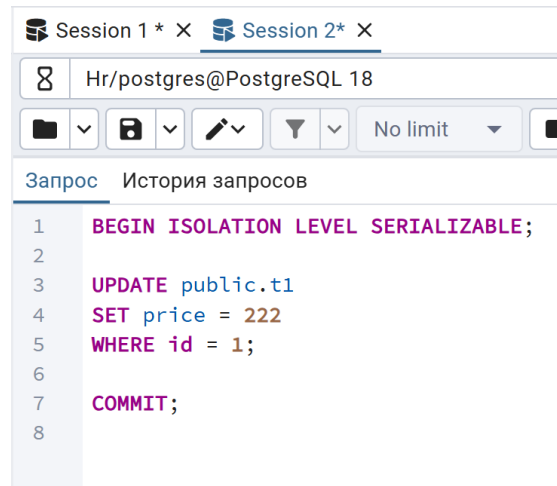


Рисунок 5.2 — Попытка обновления цены товара в Session2

Замечено, что транзакция в Session2 ожидает закрытия транзакции в Session1 так как присутствует совместная блокировка строки. Но после фиксации транзакции в Session1 блокировки на обновляемые данные снимаются, и логично ожидать, что транзакция в Session2 сможет завершиться. Однако в рамках уровня изоляции SERIALIZABLE повторное обновление одних и тех же данных в параллельных транзакциях запрещено. Поэтому при попытке выполнения транзакции в Session2 возникает ошибка, представленная на рисунке 5.3.

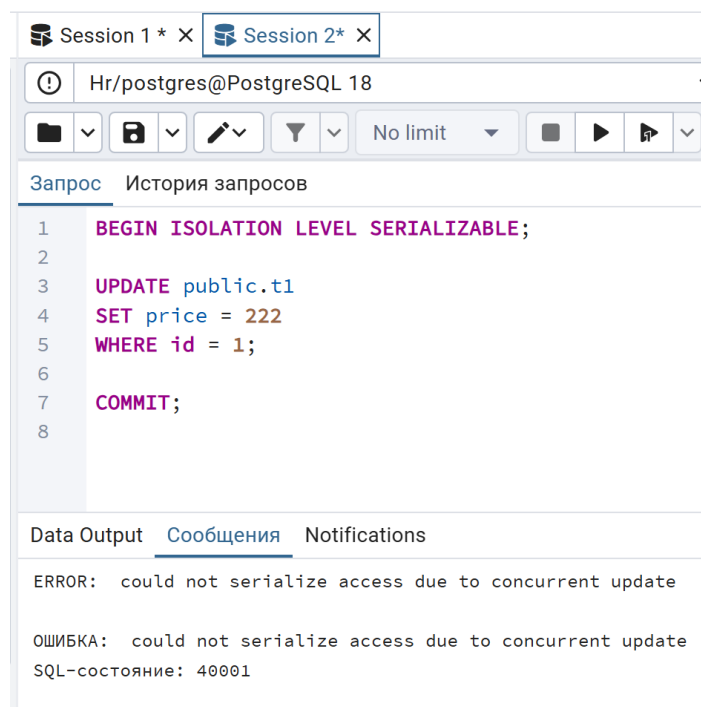


Рисунок 5.3 – Ошибка из-за уровня изоляции SERIALIZABLE

Выводы и анализ результатов работы

В ходе выполнения практической работы были выполнены все задания, предусмотренные методическими указаниями, направленные на изучение механизмов управления транзакциями, блокировками и уровней изоляции в СУБД PostgreSQL.

Были рассмотрены базовые операции управления транзакциями, такие как BEGIN, COMMIT и ROLLBACK, а также механизм точек сохранения SAVEPOINT. Было изучено то, как можно фиксировать изменения, откатывать всю транзакцию или возвращаться к промежуточному состоянию данных без завершения всей транзакции. Также в рамках работы был изучен механизм блокировок в PostgreSQL. Отдельное внимание было уделено изучению уровней изоляции транзакций. Были рассмотрены режимы READ COMMITTED, REPEATABLE READ и SERIALIZABLE. Было установлено, что уровень READ COMMITTED позволяет видеть только зафиксированные изменения, но допускает изменение результатов повторных запросов в рамках одной транзакции. Уровень REPEATABLE READ обеспечивает стабильность результатов чтения за счёт использования снимка данных на момент начала транзакции и предотвращает неповторяемое и фантомное чтение. Уровень SERIALIZABLE обеспечивает максимальную изоляцию транзакций и предотвращает логические конфликты, автоматически отменяя одну из параллельных транзакций при невозможности их сериализации.

В процессе выполнения практической работы возникли определённые сложности. Основные затруднения были связаны с одновременной работой в нескольких сессиях базы данных. На начальном этапе было сложно отслеживать, в какой именно сессии запущена транзакция, зафиксированы ли изменения и какие блокировки в данный момент удерживаются. Также дополнительные трудности вызвало понимание различий между состояниями транзакций (активная, зафиксированная, откатанная) и интерпретация информации, получаемой из системных представлений pg_stat_activity и pg_locks. Данные сложности были успешно преодолены в процессе последовательного выполнения заданий и анализа поведения транзакций. Это позволило глубже понять принципы параллельной работы транзакций и механизм обеспечения согласованности данных в PostgreSQL.

Таким образом, цель практической работы была полностью достигнута. В результате выполнения заданий были получены практические навыки работы с транзакциями, управления блокировками и анализа поведения параллельных транзакций при различных уровнях изоляции в СУБД PostgreSQL. Полученные знания могут быть использованы при проектировании и разработке надёжных многопользовательских приложений, требующих обеспечения целостности и согласованности данных.